

1142 Digital System Design Final Project

Pipelined RISC-V Design

TA Information: 王泓予

pony@access.ee.ntu.edu.tw

1. Project Description

Table 1. Required Instruction Set

Name	Description
ADD	Addition, overflow detection for signed operand is not required*
ADDI	Addition immediate with sign-extension, without overflow detection*
SUB	Subtract, overflow detection for signed operand is not required*
AND	Boolean logic operation
ANDI	Boolean logic operation with 12bit of immediate
OR	Boolean logic operation
ORI	Boolean logic operation with 12bit of immediate
XOR	Boolean logic operation
XORI	Boolean logic operation with 12bit of immediate
LLI	Shift left logical (zero padding)
SRAI	Shift right arithmetic (sign-digit padding)
SRLI	Shift right logical (zero padding)
SLT	Set less than, comparison instruction
SLTI	Set less than variable, comparison instruction
BEQ	Branch on equal, conditional branch instruction
BNE	Branch on not equal, conditional branch instruction
JAL	Unconditionally jump and link (Save next PC in \$rd)
JALR	Jump and link register(Save next PC in \$rd)
LW	Load word from data memory (assign word-aligned)
SW	Store word to data memory (assign word-aligned)
NOP	No operation(addi \$r0 \$r0 0)
FLUSH	Flush all the cache data to slow memory if there is cache in your design

* Different from the definition in [1], the exception handler **for arithmetic overflow** is not required.

In this final project, you are asked to design a **pipelined RISC-V processor (synchronous active low reset)**. This processor should at least support the instruction set defined in Table 1. The instruction set is referenced from Chapter 2 (RV32I base integer instruction set) of [1], and we encourage you to read it in detail.

The whole module hierarchy is shown in Figure 1. The processor architecture is shown in Figure 2. As you see, this is modified from the single-cycle architecture of our HW2. Your design should be **based on this 5-stage pipelined architecture** and must include **at least five pipeline stages**. You are encouraged to incorporate additional pipeline stages or apply retiming techniques to further optimize performance. You need to modify several parts to fit our specifications. For example, you need to add the path for **J-type instructions**.

The FLUSH instruction is guaranteed to be the final functional instruction in all test patterns. Once the FLUSH operation is complete, the CHIP must assert the o_done signal. The testbench will then compare the memory contents against the golden data to verify correctness.

The cache design is optional but strongly recommended to improve the system performance. In this case, your design should support the FLUSH operation. On the other hand, if there isn't any cache in your design, you can simply treat the FLUSH instruction as an NOP instruction, but the o_done signal should still be asserted.

Also, you should **solve the hazards** by adding some circuits. There are 3 hazard categories that should be properly handled in your pipelined processor:

- 1) Structure hazard
- 2) Data hazard
- 3) Control hazard

Although all of these hazards can be solved by inserting NOP manually or automatically in your test program, we ask you to implement a **data forwarding unit** and **pipeline stall unit** to solve these hazards.

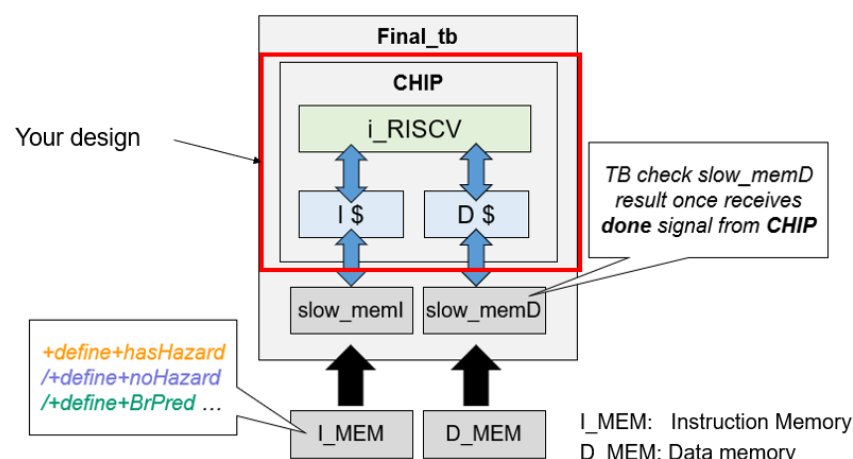


Figure 1. Module Hierarchy

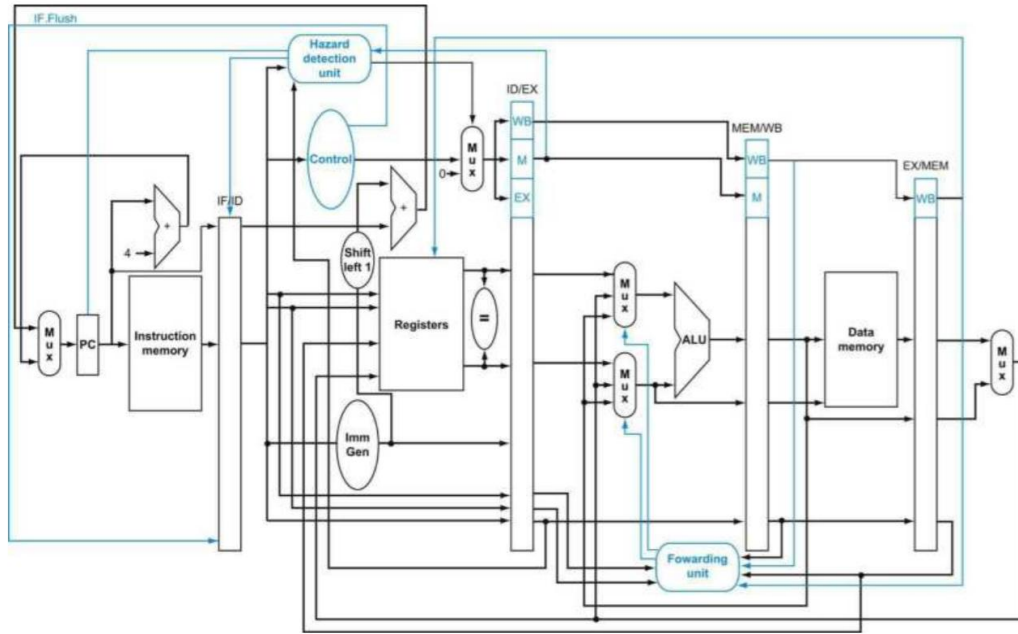


Figure 2. Simplified Pipeline Architecture of RISC-V

2. Cache and Memory Interface

The instruction memory and data memory will not be contained in your design. The memory interface is left as module I/O. You have to use the provided memory model. The design should pass all with all memory types, including `fast_memory`, `slow_memory` and `slow_memory_random_latency`. **Note that you shouldn't synthesize the memories.**

Memory type	Description
<code>fast_memory.v</code>	Fast memory without stalling
<code>slow_memory.v</code>	Slow memory with a fixed latency
<code>slow_memory_random_latency.v</code>	Slow memory with random latency

The cache design is optional. Note that we do not restrict the replacement policy and writing policy of the cache design. You are encouraged to optimize the cache units to fit your RISC-V design.

3. Synthesis Notes

You should synthesize your design using TSMC 0.13 cell library, and the relevant files, e.g. `.synopsys_dc.setup`, can be copied from previous HW or use the attached file. The design constraints are specified in “`CHIP_syn.sdc`”. You should implement your design whose top module is **CHIP**. Please do not modify the IO interface of the top module.

The post-synthesis simulation is required, and all involved Verilog files should be modeled by gate level. Note that the maximum clock frequency must be verified by post-synthesis gate-level simulation. And you are recommended to buffer the input signal to avoid timing violations.

4. Grading Policy and Possible Extensions

All grades of this project consist of two aspects:

1) Baseline checkpoint (40%)

- Checkpoint Presentation (5%)
- noHazard Gate Level (15%)
- hasHazard Gate Level (20%)

2) Final presentation and submission (60%)

- Final Presentation (8%)
- Branch Prediction Gate Level (9%)
- Compressed Instr. Gate Level (9%)
- Multiplication Instr. Gate Level (9%)
- Q_sort & Conv & LFSR Performance Ranking (15%)
- Report (10%)

Baseline Checkpoint (40%)

If your design meets all requirements of the description below and can be synthesized and simulated at the gate level, your design will be qualified to get baseline points. The solid requirements include:

- 1) Supporting all instructions above
- 2) Caches are optional but recommended
- 3) Pass test assembly programs (noHazard, hasHazard)
- 4) Complete the circuit synthesis. Note that the slack cannot be negative.
- 5) Cycle Time in Gate-level Simulation cannot be lower than the cycle time in SDC.
- 6) Cycle Time in Gate-level Simulation must below 3.5ns
- 7) PC address should be reset to 0

Also, each team must prepare a **4-6 page slide** and a **5-minute presentation** confirming your current results and future plan.

Final Presentation and Submission (60%)

There are 3 topics of extension.

- 1) Branch prediction mechanism.
- 2) Supporting compressed instructions.
- 3) Supporting multiplication instructions.

Implement the topics of extension **as much and deep as you can**; the deeper the exploration, the higher the score (The content also affects the quality of your presentation and report). Each team should prepare a full talk (within 10 minutes, about 10-15 slides) for your fantastic work on extension!

Then the performance is evaluated by Q_sort , $Conv$ and $LFSR_HIST$:

$$A \times T_{Q_sort} \times T_{Conv} \times T_{LFSR_HIST}$$

A: Area (um2)

T_{Q_sort} : Simulation time of Q_sort (ns)

T_{Conv} : Simulation time of $Conv$ (ns)

T_{LFSR_HIST} : Simulation time of $LFSR_HIST$ (ns)

5. Simulation Example

[License]

```
source /usr/cad/synopsys/CIC/vcs.cshrc
source /usr/spring_soft/CIC/verdi.cshrc
```

[Simulation]

```
// RTL (simulate under 01_RTL)
vcs ../00_TESTBED/testbench/Final_tb.v [rtl_files.v]\
../00_TESTBED/testbench/flush_inst_define.v \
../00_MEMORY/slow_memory_random_latency.v -full64 -R -debug_access+all +v2k \
+define+noHazard

// Gate level (simulate under 03_GATE)
vcs ../00_TESTBED/testbench/Final_tb.v ../02_SYN/Netlist/CHIP_syn.v \
../00_TESTBED/testbench/flush_inst_define.v \
../00_MEMORY/slow_memory_random_latency.v -v tsmc13.v -full64 -R \
-debug_access+all +v2k +define+noHazard +define+SDF
```

To change the pattern, modify the **+define+noHazard** to other pattern flag. You can find details in `../00_TESTBED/testbench/pat_define.v`

6. Submission Requirement

All the files need to be compressed as a single **ZIP file** and **uploaded to NTU COOL for each team.**

For Checkpoint Submission on 5/27 23:59:

Example of filename

DSD_Final_Check_G#_v#.zip

e.g. DSD_Final_Check_G1_v2.zip

Your submitted file should include the following files:

DSD_Final_Check_G#/

- |— *01_RTL/*
 - | |— *CHIP.v*
 - | |— *01_rtl.f*
 - | |— *[other RTL files]*
- |— *02_SYN/*
 - | |— *Netlist/*
 - | | |— *CHIP_syn.v*
 - | | |— *CHIP_syn.sdf*
 - | | |— *CHIP_syn.ddc*
- |— *DSD_Final_Checkpoint_Scores.txt*
- |— *DSD_Final_Checkpoint_Scores.pdf*
- |— *Check_Presentation.pptx*

For Final Submission on 6/10 23:59:

Example of filename

DSD_Final_G#_v#.zip

e.g. DSD_Final_G1_v2.zip

Your submitted file should include the following files:

DSD_Final_G#/

```
|— 01_RTL/  
|   |— CHIP.v  
|   |— 01_rtl.f  
|   └─ [other RTL files]  
|— 02_SYN/  
|   └─ Netlist/  
|       |— CHIP_syn.v  
|       |— CHIP_syn.sdf  
|       └─ CHIP_syn.ddc  
|— DSD_Final_Project_Scores.txt  
|— DSD_Final_Project_Scores.pdf  
└─ Check_Presentation.pptx
```

For Final Submission on 6/14 23:59:

DSD_Final_Report_G#/

Final_Presentation.pptx

Report.pdf

The homework will be graded **ONLY IF** the filename of your submission is correct!

7. Schedule and Necessary Submissions

Date	Submission/Event
5/07	Final project announcement
5/28	A. Baseline checkpoint. Each team should prepare a presentation (<i>4-6 pages, about 5 minutes</i>) to confirm your current results and future plan. B. You should upload the source codes and slide to COOL before 5/27 23:59. C. Briefly show your design for passing Baseline testbenches (noHazard, hasHazard). D. Extension topics plan should be included in the presentation E. Please attach the work assignment chart on last page
6/11	A. Final presentation. Each team should prepare a full talk (about 10-15 slides <i>within 10 minutes</i>) to demonstrate your fantastic work! Detail presentation plan will be announced on COOL. B. You should upload the source codes (including all the RTL code and synthesis related files: *.v, *.sdf, *.ddc and your performance) to COOL before 6/10 23:59.
6/14	Report submission. You should upload the report (including a detailed report (<i>8-16 pages</i>) and the presentation slides). You should upload them to COOL before 6/14 23:59.

8. Reference

[1] <https://riscv.org/specifications/>